1. What is the difference between a Task and an Operator in Airflow?

   Answer: In Airflow, an Operator represents a single, atomic task like running a SQL query, executing a bash command, or running a Python function. A Task is an instance of the operator with a specific set of input arguments. In other words, an operator defines what is to be done, and a task defines how to do it.

2. How does Airflow handle failures and retries?

   Answer: In the default_args dictionary of a DAG, you can specify the retries and retry_delay parameters. If a task fails, Airflow will wait for the duration specified in retry_delay before it attempts to execute the task again. The task will be retried until it succeeds or until the number of retries specified in retries is reached.

3. What is the purpose of the start_date and end_date in an Airflow DAG?

   Answer: The start_date specifies the first datetime for which the task should run. The end_date is an optional parameter that specifies the last datetime for which the task should run. If no end_date is specified, the task will continue to be scheduled indefinitely.

4. Can you explain the significance of Airflow's dynamic and code-driven nature?

   Answer: Airflow is code-driven, meaning workflows are defined as code, which brings advantages like versioning, code review, collaboration, and the use of any Python libraries. Its dynamic nature allows you to dynamically generate parts of your DAG based on parameters, database results, or anything else, making it flexible and adaptive to changes.

5. How does the Airflow scheduler work?

   Answer: The Airflow scheduler monitors all tasks and all DAGs to ensure that everything is being executed according to schedule or triggers. It checks the status of every task instance in the metadata database, checks dependencies, and uses this information to decide what needs to be run, and then executes the task instances once their dependencies are complete.

6. How would you ensure data quality checks within Airflow?

   Answer: You can use the CheckOperator or its subclasses like ValueCheckOperator and IntervalCheckOperator. These operators can be set

up to perform data quality checks to ensure data is correct and consistent before moving on to the next task.

7. Explain the significance of the pool parameter in tasks.

   Answer: pool is used to limit the parallelism for a particular set of tasks. This can be especially important when working with databases, to ensure that too many parallel tasks don't exhaust available connections.

8. What are XComs in Airflow?

   Answer: XComs (short for cross-communications) allow tasks to exchange messages, allowing them to send and receive values from one another. They are stored in Airflow's metadata database.

9. What is the difference between a SequentialExecutor, LocalExecutor, and CeleryExecutor in Airflow?

   Answer:
   SequentialExecutor: Runs one task instance at a time in a single process. Suitable for debugging.
   LocalExecutor: Runs tasks on the same machine in different subprocesses.
   CeleryExecutor: Uses Celery to distribute tasks to multiple worker nodes for execution, ideal for scaling out.

10. Why might you use subDAGs and what are the potential pitfalls?

    Answer: subDAGs are often used to group tasks into logical units. However, they can add complexity, especially if not set up correctly. A common mistake is not setting the subdag's schedule_interval to None, which can cause scheduling issues.

11. How does Airflow handle data lineage and tracking?

    Answer: While Airflow does not provide a native, comprehensive data lineage solution, it does allow tracking through task logs, XComs, and integrations with external tools. Some third-party tools can also be used to enhance data lineage tracking in Airflow.

12. How can you prevent a task from being executed?

    Answer: You can use the BranchPythonOperator to skip tasks based on a condition, or you can set the task_instance state to skipped programmatically.

13. How can you optimize the performance of an Airflow DAG?

    Answer: Some strategies include:
    Reduce the number of task dependencies.
    Increase parallelism settings in Airflow configurations.
    Optimize the interval of DAG runs.
    Use the right Executor for your use-case, like CeleryExecutor for distributed execution.

14. Explain the role of hooks in Airflow.

    Answer: Hooks act as interfaces to external platforms and databases, allowing tasks to interact with external systems without repeatedly handling connections and authentication.

15. How can you secure sensitive information like passwords in your Airflow DAGs?

    Answer: Airflow provides a Secrets backend which integrates with tools like HashiCorp Vault, AWS Secrets Manager, or environment variables to store and retrieve sensitive information securely.

16. How would you handle backfilling data in Airflow?

    Answer: Airflow has a backfill command which allows you to run the DAG for a specified historical period. It respects past runs, so if a task instance for a particular date has already run, it won't be rerun unless specified.

17. What's the difference between a PythonOperator and a BashOperator?

    Answer: PythonOperator is used to execute a Python function, while BashOperator is used to execute a bash command.

18. How does Airflow handle time zones with the start_date and other time-related parameters?

    Answer: As of Airflow 2.0, it supports two time zones: UTC and the system's local time zone (usually set by the AIRFLOW__CORE__DEFAULT_TIMEZONE configuration).

19. Explain the significance of catchup parameter in a DAG.

Answer: If catchup is set to True (default), when a DAG is started, it will execute all the runs that occurred since the start_date up to the current date. If set to False, it will only run the latest instance.

20. How can you share data between two operators without using a database?

    Answer: This can be done using XComs, which allows tasks to push and pull data to be used by other tasks.

21. What is DAGBag in the context of Airflow?

    Answer: DAGBag is a collection of DAGs parsed out of a folder. It's the result of Airflow scanning and parsing all the Python files in the DAG_FOLDER.

22. How can you debug a failing Airflow task?

    Answer: Reviewing the logs is the first step. Logs can be viewed via the Airflow UI or found on the worker nodes (for distributed setups). Additionally, using the test command with Airflow CLI can help to execute a task without the dependencies for debugging.

23. Describe how dynamic output types can be managed in Airflow.

    Answer: Using the BranchPythonOperator, you can determine dynamically which path or which downstream task should be executed next based on the output of a task.

24. How can you manage passwords and secrets in Airflow?

    Answer: You can use Airflow Connections feature for managing credentials or integrate with a secrets backend like HashiCorp Vault or AWS Secrets Manager.

25. How can you ensure idempotency in your Airflow tasks?

    Answer: Idempotency means even if a task runs multiple times, the outcome remains the same. You can ensure this by:
    Designing tasks such that rerunning produces the same result (e.g., always truncate a table before loading it).
    Using externally triggered runs with specific execution dates.
    Monitoring and alerting on non-idempotent tasks.

1. How would you set up dynamic parallel tasks in Airflow?

Answer: In Airflow, dynamic parallel tasks can be set up using the BranchPythonOperator and loop constructs. A common practice involves creating a Python function that generates a list of parameters. You can then loop over this list to dynamically create task instances. For instance, if you want to generate parallel tasks based on a list of cities:

```
cities = ['NYC', 'LA', 'SF']
for city in cities:
    task = PythonOperator(
        task_id=f"process_{city}",
        python_callable=process_city_function,
        op_args=[city],
        dag=dag
    )
```

2. Imagine we have a requirement to ensure that certain tasks in a DAG don't run if they don't meet specific criteria (e.g., specific date conditions). How would you implement this?

Answer: For implementing conditional tasks based on certain criteria, the BranchPythonOperator can be utilized. The function attached to this operator can check the desired criteria, and based on the outcome, decide which downstream task to run next. For example, using the SkipMixin, tasks can be skipped if they don't meet the criteria. This provides a branching mechanism where different paths in a DAG can be taken based on specific conditions.

3. Our company runs thousands of tasks every day, but our Airflow metadata database is becoming a bottleneck. How would you address this situation?

Answer: Addressing the bottleneck in the Airflow metadata database involves a multi-pronged approach:

- **Archival and Cleanup:** Archive old data or adjust the cleanup intervals to reduce the load on the database.
- **Database Scaling:** Transition to a more robust database system and consider horizontal scaling options. Database optimization, such as ensuring proper indexing and performing periodic vacuum operations, can also improve performance.
- **Configuration Adjustments:** Enable and fine-tune Airflow configurations that pertain to performance, such as increasing parallelism and concurrency limits.

4. Discuss how you would implement error handling and retries in Airflow?

Answer: Error handling and retries are essential for robust workflows. In Airflow, the retry parameter can be set when defining a task to specify how many times the task should be retried upon failure. Additionally, the retry_delay parameter can set the delay between retries. For more custom error handling, the on_failure_callback can be used to specify a function that should be called when the task fails. This function can handle logging, notifications, or any other custom error-handling logic.

5. How would you design a workflow in Airflow where data quality checks are essential, and failures in these checks should lead to notifications?

Answer: For workflows where data quality checks are paramount, one can employ the PythonOperator or CheckOperator to execute these checks. If the check identifies a quality issue, it can raise an exception, leading to the task's failure. To notify stakeholders of this failure, you can use the on_failure_callback parameter to specify a function that sends out notifications. This could be an email, a message on a platform like Slack, or any other desired notification mechanism.

6. Describe how you would use Airflow in a hybrid cloud environment where some tasks run on-premises, while others run in a public cloud.

Answer: Airflow offers a variety of operators that facilitate tasks in different environments. For on-premises tasks, operators like the SSHOperator can be used to run commands on local servers. For tasks in a public cloud, Airflow provides cloud-specific operators, such as GCPComputeStartInstanceOperator for Google Cloud or EmrAddStepsOperator for AWS EMR tasks. The key is to appropriately configure the connections in Airflow to securely connect to both on-premises and cloud environments.

7. How would you set up monitoring and logging for your Airflow setup?

Answer: Effective monitoring and logging are crucial for diagnosing issues and ensuring the health of Airflow deployments.

- **Monitoring**: Utilize Airflow's built-in web server for real-time monitoring of DAGs. Further, integrate Airflow with monitoring platforms like Grafana or Prometheus for detailed metrics and visualization.
- **Logging**: Ensure that task logs are forwarded to centralized logging solutions such as the ELK (Elasticsearch, Logstash, Kibana) stack or

Splunk. This centralization facilitates easier analysis and long-term retention.

8. How would you handle a scenario where one of your DAGs is taking a significantly longer time to execute than expected?

Answer: If a DAG is taking longer than expected, the following steps should be taken:

- **Profiling**: Examine the DAG to identify tasks that might be the bottleneck.
- **Optimization**: Refactor or optimize the tasks that are taking a long time. This might involve improving the underlying code, using more efficient algorithms, or scaling the resources available for that task.
- **DAG Splitting**: If the DAG is monolithic, consider splitting it into smaller, more manageable DAGs that can run in parallel or be scheduled differently.
- **Configuration Tweaks**: Make adjustments to the number of worker processes or threads to optimize parallel execution of tasks.

9. How would you manage and organize a large number of DAGs for different teams in an organization?

Answer: For effective management of numerous DAGs:

- **Naming Conventions**: Establish and adhere to consistent naming conventions for DAGs to easily identify their purpose and owning team.
- **Folder Organization**: Organize DAG files into structured folders based on their functionality or owning teams.
- **DAG Tags**: Use Airflow's DAG Tags feature to categorize and filter DAGs in the UI, making it easier to locate specific workflows.
- **Access Control:** Implement Role-Based Access Control (RBAC) to grant appropriate permissions and access to different teams, ensuring they can only interact with relevant DAGs.

10. Discuss how you would implement a secure data transfer between Airflow and external systems.

Answer: For secure data transfers:

- **Connections**: Leverage Airflow's Connections to securely store and manage credentials and connection details.

- **Encrypted Channels**: Always use encrypted communication channels (e.g., HTTPS, SFTP) when interacting with external systems.
- **Secret Management**: Consider integrating Airflow with secret management tools such as HashiCorp's Vault for an added layer of security and centralized management of sensitive data.

Airflow Assignment: GCP Dataproc PySpark Job

Objective: Automate a workflow using Apache Airflow to process daily incoming CSV files from a GCP bucket using a Dataproc PySpark job and save the transformed data into a Hive table.

Tasks:

1. **Setup:**
    - Create a Google Cloud Platform (GCP) bucket to store the daily CSV files.
    - Set up an Apache Airflow environment and ensure GCP and Dataproc plugins/hooks are available.
2. **DAG Configuration:**
    - Create a new DAG gcp_dataproc_pyspark_dag.
    - Schedule the DAG to run once a day.
    - Ensure catchup is set to False: catchup=False.
3. **File Sensor Task:**
    - Add a GCSObjectExistenceSensor task to check for the presence of the daily CSV file in the GCP bucket.
    - Configure the task to poke for the file every 5 minutes for a maximum of 12 hours.
4. **Dataproc Cluster Creation Task:**
    - Use the DataprocClusterCreateOperator to create a new Dataproc cluster.
    - Define and configure the cluster specifications as needed.
5. **PySpark Job Execution Task:**

- Upload your PySpark script to GCP (either in a bucket or Cloud Storage).
- Use the DataProcPySparkOperator to execute the PySpark script on the created Dataproc cluster.
- The PySpark script should:
    - Read the daily CSV file from the GCP bucket.
    - Perform some logical transformations on the data.
    - Write the transformed data into a Hive table.

6. **Dataproc Cluster Deletion Task:**

    - Use the DataprocClusterDeleteOperator to delete the Dataproc cluster once the PySpark job is successfully completed.

7. **DAG Dependency Configuration:**

    - Set the task dependencies using the set_upstream and set_downstream methods or the bitshift operators (>> and <<).
    - Ensure that the DAG tasks run in the correct sequence.

Evaluation Criteria:

- Proper configuration and structuring of the Airflow DAG.
- Successful execution and scheduling of the DAG.
- Correct sensing of the daily CSV file.
- Successful creation and deletion of the Dataproc cluster.
- Successful execution of the PySpark job with the desired transformation.
- Proper writing of the transformed data to the Hive table.

Tips:

- Remember to configure the necessary GCP connection in the Airflow web UI.
- Ensure you handle exceptions and potential issues in the workflow, such as cluster creation failures or script execution errors.
- Log important steps and outputs for easier debugging.

Submission:

- Submit the DAG python file (gcp_dataproc_pyspark_dag.py).
- Provide a brief report explaining the workflow, any challenges faced, and their solutions.
- Include screenshots of the successful DAG runs and the resulting data in the Hive table.

Name: Abhishek Obla Hema
Email: oh_abhishek@yahoo.com

Airflow Assignment-1 Documentation

This file details and describes all the attached files for the Airflow Assignment -1

Tools Used:

1. Python3 – Microsoft VScode
2. Apache Airflow
3. GCP services – DataProc/GCS
4. Apache Hive

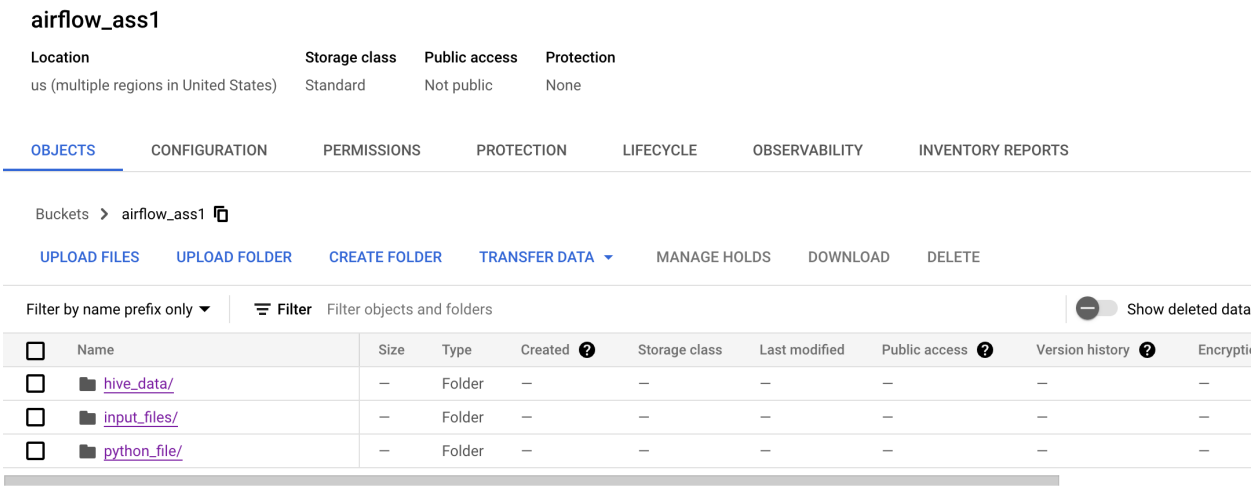
Files Attached:

1. Airflow_assignment_1.pdf – This file
2. Employee_batch.py – Pyspark job that filters employees with salary >=60,000 and stores it in a GCS location.
3. Airflow_ass1_job.py – Airflow job that details the DAGs
4. Employee.csv – Csv file used for the exercise

Process and File Descriptions:

Step 1:

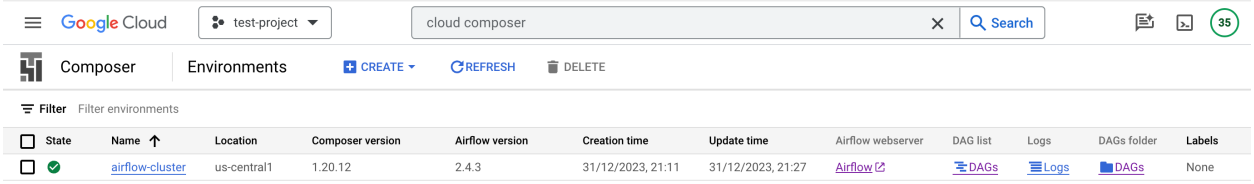
I created a bucket called 'airflow_ass1' and placed employee.csv under input_files folder. This file will be picked up by the spark job which is a part of the DAG

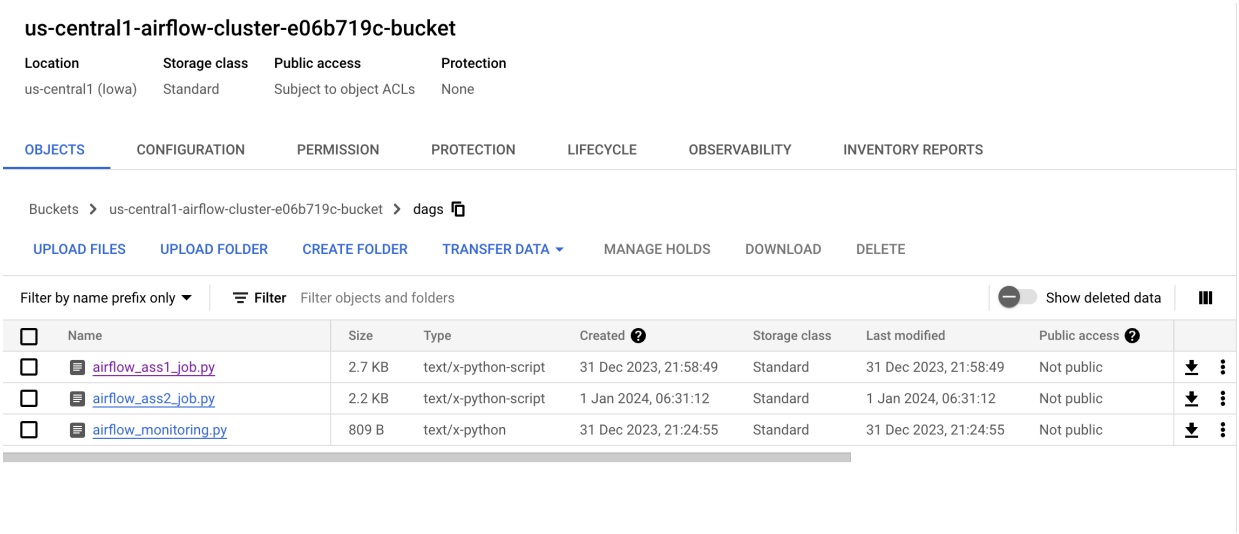


I also made sure to place the pyspark job 'employee_batch.py' in the python_file folder in the same GCS bucket.

Step 2:

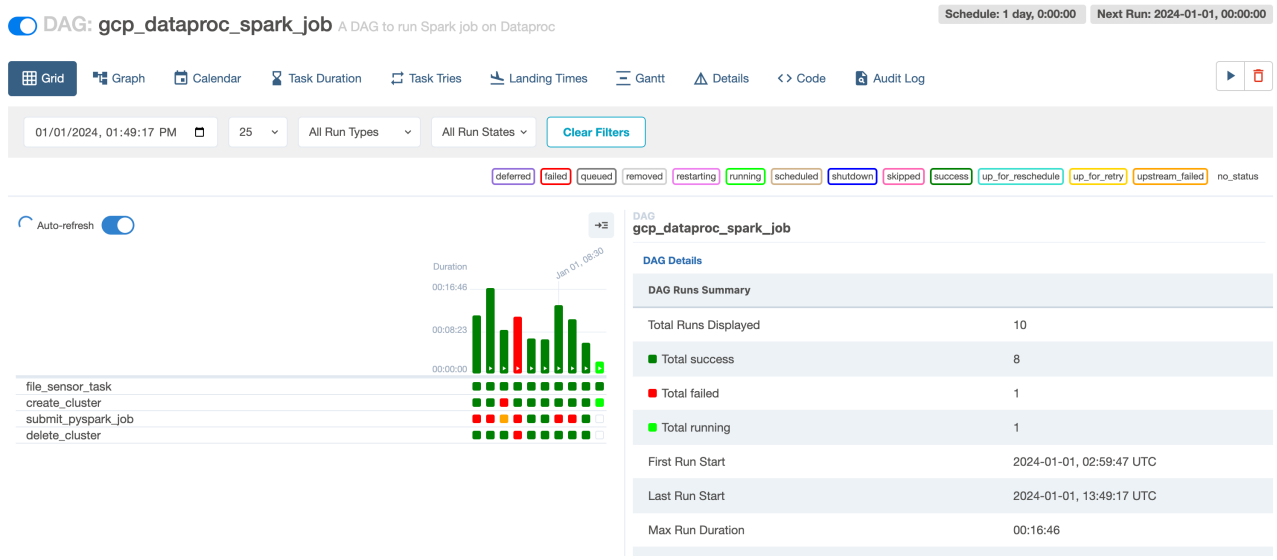
I create an airflow cluster and then proceeded to place the ‘airflow_ass1_job.py’ file in the DAG list so that it can be picked up by airflow

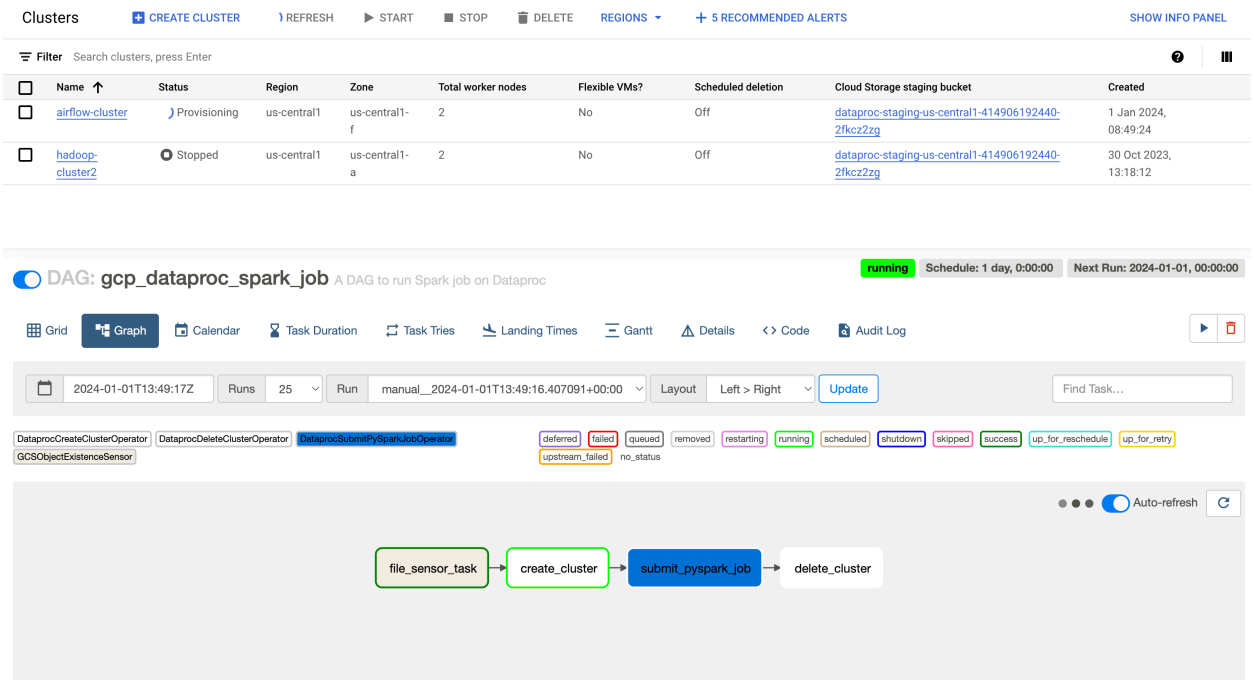




Step 3:

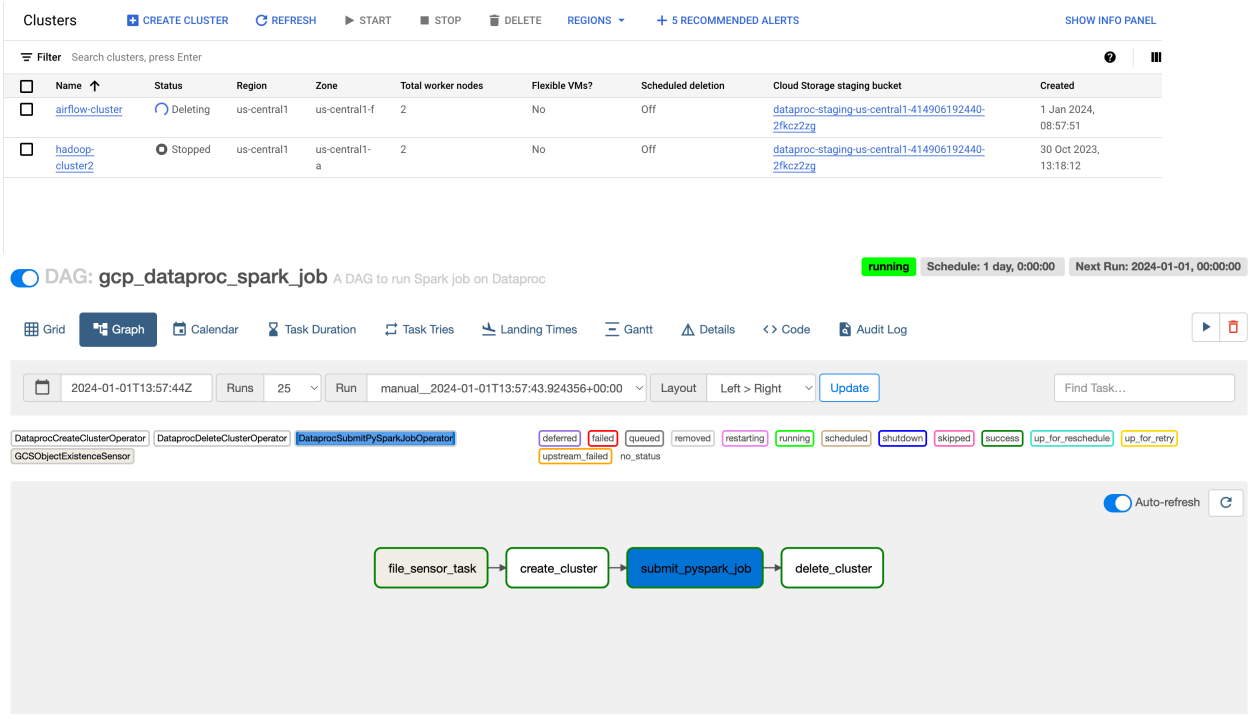
Now we can see the various stages in the DAG. A file sensor checks every 5 mins in the input_file location, once it detects employee.csv a new cluster is created followed by which the spark job is launched which then filters employees with salary >=60,000 and then places the output in another GCS location in the bucket airflow_ass1 under the hive_data folder.





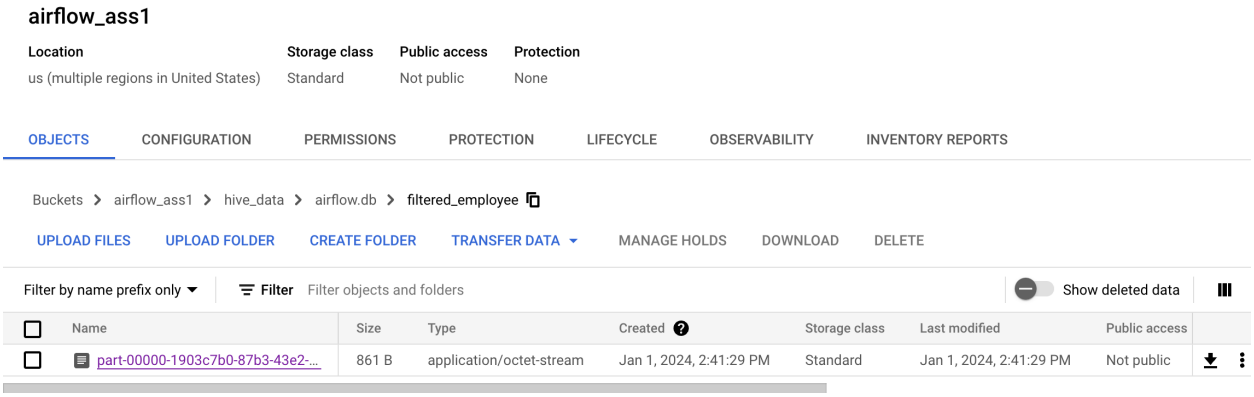
Step 4:

We can see that all stages have run successfully followed by deletion of the cluster as well



Step 5:

We can see the resultant data saved in a parquet file format saved in hive_data below (over which we can build external hive tables to query the data)



Challenges

1. Issues with the spark job being picked up since we had to define a location to save the output.
2. We can't put the output on the local of the cluster since with the deletion phase of the cluster this data would disappear as well. Hence it makes sense to place the output in a GCS bucket
3. Make sure to specify the format like "hive"/"parquet" while saving the output.

Assignment: Implement an Airflow DAG to Load Daily JSON Data from GCP Bucket to Hive on Dataproc

Objective: Create an Apache Airflow DAG that fetches a daily JSON file from a GCP bucket and appends the data into a Hive table on GCP Dataproc.

1. Airflow Setup:

   - Ensure that the necessary Airflow providers for Google Cloud Platform and Hive are installed.

2. Airflow Connections:

   - GCP Connection: Set up a connection in the Airflow UI for accessing the GCP bucket.

   - Hive Connection: Set up a connection in Airflow UI to interface with Hive on GCP Dataproc.

3. DAG Implementation:

   - DAG Initialization: Define a DAG with a daily schedule_interval.

   - Operators:
     - Use GCSToLocalFilesystemOperator to download the daily JSON file from the GCP bucket to the local Airflow directory.
     - Use HiveOperator to load the downloaded JSON data into the Hive table on Dataproc.

- Task Dependencies: Ensure the correct execution order for the tasks.

4. Testing:

- DAG Execution: Trigger your DAG manually and verify its successful execution.

- Data Validation: Query the Hive table on Dataproc to ensure that the data from the JSON file is correctly appended.

5. Submission:

- Provide the DAG Python script.
- Write a brief report on your implementation steps and any challenges faced.

Name: Abhishek Obla Hema
Email: oh_abhishek@yahoo.com

Airflow Assignment-2 Documentation

This file details and describes all the attached files for the Airflow Assignment -2

Tools Used:

1. Python3 – Microsoft VScode
2. Apache Airflow
3. GCP services – DataProc/GCS
4. Apache Hive

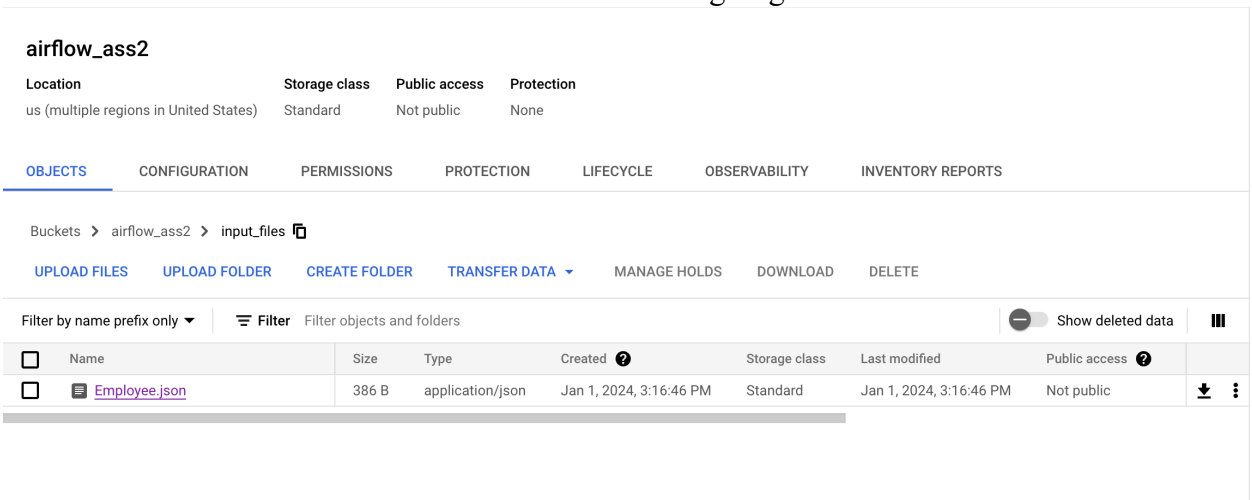
Files Attached:

1. Airflow_assignment_2.pdf – This file
2. Airflow_ass2_job.py – Airflow job that details the DAGs
3. Employee.json – JSON file used for the exercise

Process and File Descriptions:

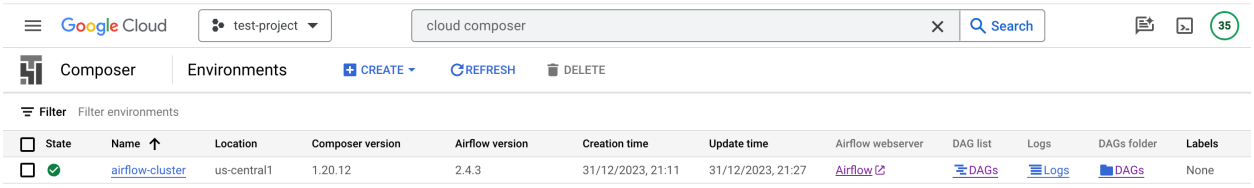
Step 1:

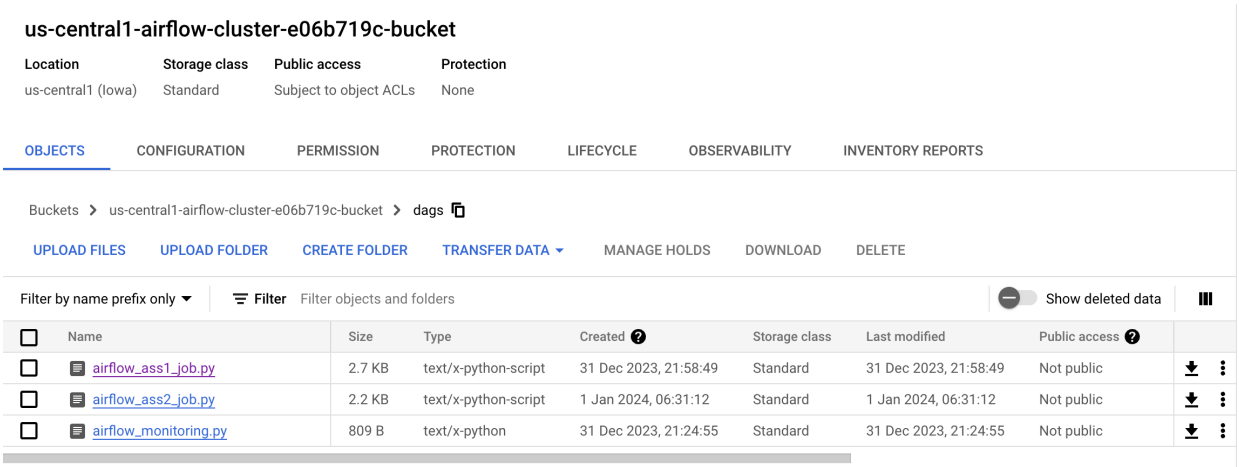
I created a bucket called 'airflow_ass2' and placed Employee.json under input_files folder. This file will be downloaded to the airflow local before being staged for an external hive table.



Step 2:

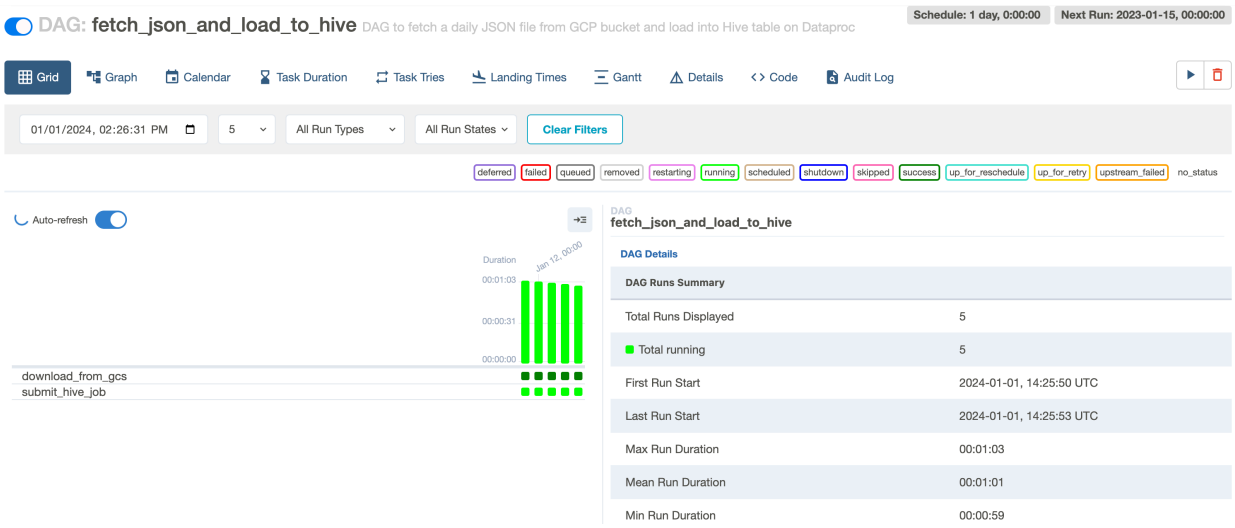
I create an airflow cluster and then proceeded to place the 'airflow_ass2_job.py' file in the DAG list so that it can be picked up by airflow





Step 3:

Now we can see the various stages in the DAG. The first task is about downloading the json file from the GCS bucket to the airflow local. It then stages this json file in a local directory on the airflow cluster itself over which external tables can be built.



Challenges

1. The HiveOperator is deprecated and hence had to use the new DataProcSubmitJobOperator
2. Learnt the hard way that file_names are case sensitive so make sure to refer to the path and files with correct case.
3. Difficult overall to troubleshoot the leading cause when airflow jobs fail